ShadowTrace Website Security Audit Report

Date: 28 February 2026

Scope: shadowtrace.ai — Express.js server, chatbot API, client-side code, dependencies

Classification: Confidential

Executive Summary

This report documents the findings of a security review of the ShadowTrace marketing website codebase, covering the Express.js server (server.js), chatbot API routes, client-side JavaScript, dependency chain, and secrets management. The review identified **1 critical**, **3 high**, **3 medium**, and **2 low** severity findings.

Severity	Count	Key Risk
CRITICAL	1	Live OpenAI API key in .env files (not gitignored correctly)
HIGH	3	Prompt injection, server.js exposes all files, no security headers
MEDIUM	3	No CORS on main server, no CSRF protection, rate limit bypass
LOW	2	qs DoS vulnerability, in-memory rate limit (no persistence)

Detailed Findings

Finding 1 — CRITICAL: Live OpenAI API Key Committed to Repository

File: .env, chatbot-api/.env

Both .env files contain a live OpenAI API key (sk-proj-pgTT...pXGIA). While .env is listed in .gitignore and is not tracked in git, the key is present in the working directory on the Railway deployment. If the deployment environment exposes the filesystem (e.g., via a misconfigured debug endpoint, directory traversal, or backup leak), this key would be compromised.

Additionally, the .env.example files — which ARE tracked in git — contain the same live key value instead of a placeholder. Anyone with access to the repository can see the real API key in .env.example.

Impact: An attacker with access to the key could make unlimited OpenAI API calls billed to your account, potentially costing thousands of pounds. They could also extract the system prompt and manipulate the chatbot.

Remediation:

1. Immediately rotate the OpenAI API key at platform.openai.com. 2. Replace the live key in .env.example with a placeholder (e.g., sk-proj-YOUR_KEY_HERE). 3. Scrub git history using git filter-branch or BFG Repo-Cleaner if .env.example was committed with the real key. 4. Set API keys exclusively via Railway environment variables, not .env files on disk.

Finding 2 — HIGH: Static File Server Exposes Entire Project Directory

File: server.js, line 225

express.static(path.join(__dirname)) serves the entire project root as static files. This means server.js itself, package.json, package-lock.json, .env.example (containing the real API key), the chatbot-api/ directory, and any other file in the project root is publicly accessible via HTTP.

An attacker can directly request https://shadowtrace.ai/server.js, /package.json, /.env.example, or /chatbot-api/routes/chat.js and read your server-side source code, dependencies, and configuration.

Impact: Full source code disclosure, API key exposure (via .env.example), system prompt leakage, and dependency version information useful for targeted attacks.

Remediation:

Move all public-facing static files (HTML, CSS, JS, assets) into a dedicated /public directory and change the static middleware to: express.static(path.join(__dirname, 'public')). Server-side files (server.js, package.json, .env, chatbot-api/, node_modules/) must never be in the static serving path.

Finding 3 — HIGH: Chatbot Prompt Injection Vulnerability

File: server.js line 154, chatbot-api/routes/chat.js line 65

User messages from the chatbot are passed directly to the OpenAI API with no sanitisation or filtering. An attacker can craft messages that instruct the model to ignore its system prompt, reveal the full system prompt contents (including pricing details, competitor positioning, and internal personas), or generate harmful/misleading content attributed to ShadowTrace.

The system prompt contains sensitive business intelligence: exact pricing (from ~£500/month), competitor comparisons (Chainalysis, Elliptic, TRM Labs pricing), internal persona names, and sales strategy. All of this is extractable via prompt injection.

Impact: Competitor intelligence leakage, brand reputation risk, potential misuse of chatbot to generate misleading statements.

Remediation:

1. Add input sanitisation: strip or reject messages containing instruction-override patterns (e.g., 'ignore previous instructions', 'system prompt', 'you are now'). 2. Add a guardrail layer that checks model output for system prompt leakage before returning to the client. 3. Move sensitive pricing and competitor data out of the system prompt and into a retrieval layer that the model can query but cannot directly echo. 4. Consider using OpenAI's moderation API as a pre-filter on user input.

Finding 4 — HIGH: No Security Headers (Helmet)

File: server.js

The Express server sets no HTTP security headers. Missing headers include Content-Security-Policy, X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, X-XSS-Protection, and Referrer-Policy. This leaves the site vulnerable to clickjacking, MIME-type sniffing attacks, and makes XSS exploitation easier if an injection point is found.

Impact: Clickjacking attacks (embedding shadowtrace.ai in a malicious iframe), easier exploitation of any future XSS vulnerabilities, no HSTS enforcement.

Remediation:

Install and configure the helmet middleware: npm install helmet, then add app.use(helmet()) near the top of server.js. Configure Content-Security-Policy to restrict script sources to 'self' plus your CDN domains (cdn.tailwindcss.com, code.iconify.design, fonts.googleapis.com).

Finding 5 — MEDIUM: No CORS Policy on Main Server

File: server.js (main)

The main server.js has no CORS configuration, meaning the /api/chat and /api/lead endpoints accept requests from any origin. The chatbot-api/server.js correctly implements CORS with an allowlist, but the main server — which is the one actually deployed on Railway — does not. Any website can make API calls to your chat and lead endpoints.

Impact: An attacker could build a page that calls /api/chat from their domain, consuming your OpenAI API credits. They could also spam /api/lead with fake leads.

Remediation:

Install cors and configure it with an allowlist of your domains: ['https://shadowtrace.ai', 'https://www.shadowtrace.ai']. Apply it specifically to API routes.

Finding 6 — MEDIUM: No CSRF Protection on Forms

File: contact.html, chatbot.js

The contact form and chatbot lead capture form submit to /api/lead with no CSRF token. While the form is same-origin, the lack of CORS restrictions (Finding 5) means a third-party site could craft a POST request that submits fake leads or triggers unwanted chatbot conversations on behalf of a visiting user.

Remediation:

Implement CORS restrictions (Finding 5) as the primary defence. Optionally add a CSRF token for the contact form via a server-rendered token or the double-submit cookie pattern.

Finding 7 — MEDIUM: Rate Limiter Bypassable via IP Spoofing

File: server.js, lines 14, 127

The server uses trust proxy with a value of 1, meaning it trusts the X-Forwarded-For header from the first proxy. Rate limiting is based on req.ip (derived from this header). An attacker can bypass the rate limit by rotating the X-Forwarded-For header value with each request, getting effectively unlimited API calls.

Impact: Rate limit bypass leading to OpenAI API credit exhaustion and potential denial of service.

Remediation:

1. Use a production rate limiter like express-rate-limit with a Redis or Memcached store. 2. Verify Railway's proxy behaviour and configure trust proxy accordingly. 3. Consider adding API key authentication or a simple proof-of-work challenge for the chat endpoint. 4. Set a hard spending cap on the OpenAI account as a financial safeguard.

Finding 8 — LOW: qs Dependency Vulnerability (CVE)

Source: npm audit

The qs package (versions 6.7.0-6.14.1) used by Express has a known denial-of-service vulnerability via arrayLimit bypass in comma parsing (GHSA-w7fw-mjwx-w883). This is a transitive dependency through Express.

Remediation: Run npm audit fix to update to a patched version.

Finding 9 — LOW: In-Memory Rate Limiting (No Persistence)

File: server.js, chatbot-api/routes/chat.js

Rate limiting uses an in-memory Map that resets on every server restart or redeployment. Additionally, the Map is never cleaned up, which could theoretically grow unbounded over time (minor memory leak). In a multi-instance deployment, each instance has its own independent rate limit store.

Remediation: Use express-rate-limit with a Redis store for persistent, shared rate limiting across instances.

Positive Findings

XSS Protection in Chatbot: The chatbot widget uses a proper escapeHtml() function that creates a text node and reads innerHTML, correctly escaping user input before rendering. This prevents stored XSS via chat messages.

JSON Body Size Limit: express.json({ limit: '10kb' }) restricts request body size, mitigating large-payload denial-of-service attacks.

Message Role Validation: The /api/chat endpoint validates that message roles are only 'user' or 'assistant', preventing system-role injection via the API.

Conversation History Limit: Messages are sliced to the last 10, preventing token-stuffing attacks that could inflate OpenAI API costs.

.env Files Not in Git: The .env files are correctly excluded from version control via .gitignore.

OpenAI Error Handling: API errors are caught and returned as generic messages, not leaking stack traces or internal details to clients.

Max Tokens Capped: OpenAI responses are capped at 500 tokens, limiting cost exposure per request.

Remediation Priority

Priority	Finding	Effort	Action
1 (Now)	F1: API Key Exposure	10 min	Rotate key, fix .env.example, set via Railway env vars
2 (Now)	F2: Static file exposure	30 min	Move public files to /public dir, update express.static path
3 (Today)	F4: Security headers	15 min	npm install helmet; app.use(helmet())
4 (Today)	F5: CORS policy	15 min	npm install cors; configure allowlist for API routes
5 (This week)	F3: Prompt injection	2-4 hrs	Input filter + output guardrail + move sensitive data
6 (This week)	F7: Rate limit bypass	1 hr	Switch to express-rate-limit + Redis; add OpenAI spend cap
7 (Next sprint)	F6: CSRF protection	1 hr	Double-submit cookie or token-based CSRF
8 (Next sprint)	F8: qs vulnerability	5 min	npm audit fix
9 (Backlog)	F9: Persistent rate limit	30 min	Redis-backed rate limit store

---

Report prepared by Claude for Stephen Leathem, ShadowTrace Ltd. This audit covers static analysis of the codebase only and does not include dynamic penetration testing, infrastructure review, or third-party service configuration.